Fall 2026 CS 18000-PIN – Practice Midterm 2 Comprehensive Guide

Q1: Hello File

Challenge Description

Write a program that creates hello.txt holding exactly these three lines:

```
None
Hello, file!
This is CS 180.
Goodbye.
```

Each line ends with a line break, including the last. If hello.txt already exists, the program replaces its contents. The program prints nothing to the screen. Main.java holds an empty main; the whole program is yours to write.

Sample

After the program runs, hello.txt holds the three lines above and nothing else. Press *Test* to check your work. The sample above is one of the tests and is worth no points; the rest are hidden.

Q2: Ticket

Challenge Description

Implement the class Ticket in Ticket.java; the file holds the signatures, but nothing else. A ticket knows its event (a String) and its price (a double). All fields are private.Ticket(String event, double price) stores both and counts the new ticket.getEvent() and getPrice() return the fields; setPrice(double price) changes the price.static int getCount() returns how many Ticket objects have been created since the program started.Main.java is a small program that uses your class: do not edit Main.java.

Sample

Java

```
None
Ticket t = new Ticket("Hoops", 12.0);
Ticket u = new Ticket("Play", 8.5);
t.getEvent()        ->  "Hoops"
t.getPrice()        ->  12.0
```

```
Ticket.getCount()   ->  2
t.setPrice(15.0);
t.getPrice()        ->  15.0
```

Press *Test* to check your work.

Provided Code Template (Ticket.java)

Java

```
None
/**
 * One ticket for an event. All fields are private.
 */
public class Ticket {
    /** Creates a ticket for the event at this price, and counts
it. */
    public Ticket(String event, double price) {
    }
    /** The event's name. */
    public String getEvent() {
        return null;
    }
    /** The ticket's price. */
    public double getPrice() {
        return 0.0;
    }
    /** Changes the price. */
    public void setPrice(double price) {
    }
    /** How many tickets have been created since the program
started. */
    public static int getCount() {
        return 0;
    }
}
```

Q3: Volume

Challenge Description

Write the class Volume in Volume.java; the file holds the signatures, nothing else. All fields are private. A speaker's volume is a level from 0 to 10. setLevel(int level) stores the value only if it is in range; an out-of-range value changes nothing. The constructor Volume(int level) sends its argument through setLevel, so a bad starting value leaves the level at 0.getLevel() returns the level. up() raises the level by 1 unless it is already 10; down() lowers it by 1 unless it is already 0.Main.java is a small program that uses your class: do not edit it.

Sample

Java

```
None
Volume v = new Volume(4);
v.up();
v.getLevel()       ->  5
v.setLevel(42);
v.getLevel()       ->  5
new Volume(15).getLevel()   ->  0
```

Press *Test* to check your work.

Provided Code Template (Volume.java)

Java

```
None
/**
 * A speaker volume from 0 to 10. All fields are private.
 */
public class Volume {
    /** Starts at the given level if it is 0 to 10, otherwise at 0.
*/
    public Volume(int level) {
    }
    public int getLevel() {
        return 0;
    }
    /** Stores the level only if it is 0 to 10; otherwise nothing
changes. */
    public void setLevel(int level) {
    }
    /** One louder, but never past 10. */
    public void up() {
```

```
    }
    /** One quieter, but never below 0. */
    public void down() {
    }
}
```

Q4: Budget

Challenge Description

Files given to you (do not edit these):

- Payable.java : an interface with two promises, String payTo() and double weeklyPay().
- Coach.java and Building.java implement it: a coach is paid 2400.0 and pays to the coach's name; a building is paid 800.0 and pays to Physical Facilities.
- Main.java is a small program that uses your class. Do not change the given files.

Your task: Write Budget.java, which contains two static methods that work for **any** Payable, including classes you have never seen:

- static double total(Payable[] payroll) returns the sum of every weeklyPay().
- static String biggest(Payable[] payroll) returns the payTo() of the element with the largest weeklyPay(); on a tie the first one wins.
- An empty array gives 0.0 and the empty string "". A weeklyPay() may be negative (a refund).

Sample

Java

```
None
Payable[] payroll = { new Coach("Knuth"), new Building("Matthews")
};
Budget.total(payroll)      ->  3200.0
Budget.biggest(payroll)    ->  "Knuth"
```

Press *Test* to check your work.

Provided Code Template (Budget.java)

Java

```
None
/**
```

```
 * Totals and compares anything Payable.
 */
public class Budget {
    /** The sum of every weeklyPay() in the array; 0.0 for an empty
array. */
    public static double total(Payable[] payroll) {
        return 0.0;
    }
    /** The payTo() of the largest weeklyPay(); the first on a tie;
"" for an empty array. */
    public static String biggest(Payable[] payroll) {
        return null;
    }
}
```

Q5: Ticket toString

Challenge Description

Ticket.java is complete except for one method. A ticket has an event, a price and a quantity, with getters and setPrice.

Write toString() so that it returns exactly Ticket[Hoops: $12.00 x 3]: the word Ticket, a bracket, the event, a colon and a space, the price with a dollar sign and exactly two decimals, x, the quantity, and the closing bracket. The description always shows the object as it is *now*, after any changes.

Main.java is a small program that uses the class: do not change it.

Sample

Java

```
None
Ticket t = new Ticket("Hoops", 12.0, 3);
t.toString()     ->  "Ticket[Hoops: $12.00 x 3]"
t.setPrice(20.0);
"" + t           ->  "Ticket[Hoops: $20.00 x 3]"
```

Press *Test* to check your work.

Provided Code Template (**Ticket.java**)

Java

```
None
/**
 * A ticket: an event, a price and a quantity. Everything is
written except toString.
 */
public class Ticket {
    private String event;
    private double price;
    private int quantity;

    public Ticket(String event, double price, int quantity) {
        this.event = event;
        this.price = price;
        this.quantity = quantity;
    }

    public String getEvent() {
        return event;
    }

    public double getPrice() {
        return price;
    }

    public int getQuantity() {
        return quantity;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    /** For example Ticket[Hoops: $12.00 x 3]. */
    @Override
    public String toString() {
        return null;
    }
```

```
}
```

Q6: Reservation

Challenge Description

Customer.java is given to you already completed: Customer(String name, boolean member) with getName(), isMember() and setMember(boolean member). Main.java is a small program that uses the classes here. Do not change either.

Implement Reservation.java; the file contains only the signatures.

All fields are private. Reservation(Customer customer, int nights, double rate) stores all three fields. getCustomer() and getNights() return the fields; setNights(int nights) changes the nights.

cost() is nights x rate, and a customer who is a member pays 10% less; the reservation asks its customer whether they are a member **each time** cost() is called, so a change of membership changes the cost.

toString() returns exactly Reservation[Ada, 3 nights, $270.00]: the customer's name, the number of nights followed by the word nights (also for 1), and the cost with a dollar sign and exactly two decimals.

Sample

Java

```
None
Customer ada = new Customer("Ada", true);
Reservation r = new Reservation(ada, 3, 100.0);
r.cost()         ->  270.0
r.toString()     ->  "Reservation[Ada, 3 nights, $270.00]"
ada.setMember(false);
r.cost()         ->  300.0
```

Press *Test* to check your work.

Provided Code Template (Reservation.java)

Java

```
None
/**
 * A hotel reservation for a customer. All fields are private.
```

```
 */
public class Reservation {
    public Reservation(Customer customer, int nights, double rate)
{
    }
    public Customer getCustomer() {
        return null;
    }
    public int getNights() {
        return 0;
    }
    public void setNights(int nights) {
    }
    /** nights x rate; a member pays 10% less. Asks the customer
each time. */
    public double cost() {
        return 0.0;
    }
    /** For example Reservation[Ada, 3 nights, $270.00]. */
    @Override
    public String toString() {
        return null;
    }
}
```

Q7: Robust Quantity

Challenge Description

Ask Enter quantity: until the user types a whole number from 1 to 99. For any input that is not a whole number, print Not a number.; a whole number outside 1 to 99 receives Out of range..

When a quantity is accepted, print Quantity: followed by the number.

Keep asking until the input is accepted. Every prompt counts as one attempt, including the one that is accepted. After accepting, print the result line and then Attempts: followed by the number of attempts. The program must never crash, whatever is typed.

Sample

Plaintext

```
None
Enter quantity:
[abc]
Not a number.
Enter quantity:
[150]
Out of range.
Enter quantity:
[12]
Quantity: 12
Attempts: 3
```

Press *Test* to check your work.

Provided Code Template (Main.java)

Java

```
None
void main() {
}
```

Q8: Overnight Lows

Challenge Description

lows.txt holds one overnight low temperature per line: a whole number of degrees, possibly negative. Read the file and write summary.txt with exactly these four lines, in this order:

- Nights: followed by how many lines there were;
- Average: followed by the average with exactly two decimals;
- Coldest: followed by the smallest value;
- Below freezing: followed by how many nights were strictly below 32.

The input file is in your workspace. If the input file cannot be opened, print exactly one line to the screen, Cannot open lows.txt., write no report file at all, and do not crash.

Main.java holds an empty main; your program must be written in this file.

Sample

Plaintext

```
None
lows.txt:                summary.txt:
```

```
31                      Nights: 7
28                      Average: 23.57
35                      Coldest: -4
-4                      Below freezing: 5
40
12
23
```

Press *Test* to check your work.

Provided Code Template (Main.java)

Java

```
None
void main() {
}
```

Q9: Manager

Challenge Description

Employee.java is given, complete: Employee(String name, double rate) with getName(), getRate(), double pay(double hours) (rate × hours) and toString() (Ada ($20.0/hr)). It has no zero-argument constructor. Main.java is a small program that uses the classes that you can use for testing. Do not change these files.

Write Manager.java: a manager *is an* employee with a weekly bonus. Manager(String name, double rate, double weeklyBonus); getWeeklyBonus(); pay(hours) is the employee pay for those hours plus the bonus, added once, whatever the hours; toString() is the employee description followed by (manager).

As given, your file does not compile until its constructor calls the parent's constructor. The override must build on the parent's answer by calling super.pay(...), and toString() must build on super.toString(); the tests check these by recompiling your class against a differently written parent. All new fields are private.

Sample

Java

```
None
Manager m = new Manager("Ada", 20.0, 150.0);
m.getName()       ->  "Ada"
```

```
m.pay(10)         ->  350.0
m.pay(0)          ->  150.0
m.toString()      ->  "Ada ($20.0/hr) (manager)"
Employee e = m;
e.pay(10)         ->  350.0
```

Press *Test* to check your work.

Provided Code Template (Manager.java)

Java

```
None
/**
 * A manager is an employee with a weekly bonus.
 */
public class Manager extends Employee {
    public Manager(String name, double rate, double weeklyBonus) {
    }
    public double getWeeklyBonus() {
        return 0.0;
    }
    /** The employee pay for the hours, plus the bonus once. */
    @Override
    public double pay(double hours) {
        return 0.0;
    }
    /** The employee description followed by " (manager)". */
    @Override
    public String toString() {
        return null;
    }
}
```

Q10: Roster

Challenge Description

Person.java, Student.java and Staff.java are given and read-only: Person(String name) with getName(); Student extends Person with getMajor(); Staff extends Person with getJobTitle() and

getSalary(). The tests also use subclasses you have never seen. Main.java is a small program that uses the classes.

Write Roster.java, two static methods:

- static String describe(Person p) returns "Student of CS" for a student (the word Student of and the major), Staff: Registrar ($52000.00) for staff (Staff: , the job title, and the salary in parentheses with a dollar sign and exactly two decimals), and Person for anyone else.
- static int countStudents(Person[] room) returns how many elements of the array are students.

Sample

Java

```
None
Person ada = new Student("Ada", "CS");
Person knuth = new Staff("Knuth", "Registrar", 52000.0);
Person sam = new Person("Sam");

Roster.describe(ada)     ->  "Student of CS"
Roster.describe(knuth)   ->  "Staff: Registrar ($52000.00)"
Roster.describe(sam)     ->  "Person"
Roster.countStudents(new Person[] { ada, knuth, sam, new
Student("Bo", "MATH") })   ->  2
```

Press *Test* to check your work.

Provided Code Template (Roster.java)

Java

```
None
/**
 * Describes and counts the people in a room. The classes Person,
Student and Staff are read-only.
 */
public class Roster {
    /** "Student of CS", "Staff: Registrar ($52000.00)", or
"Person" for anyone else. */
    public static String describe(Person p) {
        return null;
```

```
    }
    /** How many elements of the room are students. */
    public static int countStudents(Person[] room) {
        return 0;
    }
}
```

General Java Quick Reference

Input and Output

- **IO.readln(p)** (Returns: String) – prints the prompt p, then reads one line of input
- **IO.readln()** (Returns: String) – reads one line of input (no prompt)
- **IO.println(t)** (Returns: void) – prints t, then ends the line
- **IO.println()** (Returns: void) – prints a blank line
- **IO.print(t)** (Returns: void) – prints t without ending the line

Static Methods and Constants

- **Integer.parseInt(s)** (Returns: int) – the whole number written in s
- **Double.parseDouble(s)** (Returns: double) – the decimal number written in s
- **String.format("%.2f", x)** (Returns: String) – x with 2 decimals (%.1f for 1)
- **String.format("%s: $%.2f", t, x)** (Returns: String) – each % hole filled in order: %s a String, %d an int, %.2f a double
- **Math.abs(x)** (Returns: same type as x) – the absolute value
- **Math.min(x, y)** (Returns: same type as arguments) – the smaller
- **Math.max(x, y)** (Returns: same type as arguments) – the larger
- **Math.PI** (Returns: double) – 3.141592653589793
- **Objects.equals(a, b)** (Returns: boolean) – true if both are null, false if exactly one is, otherwise a.equals(b) (needs import java.util.Objects;)

Strings

- **s.length()** (Returns: int) – how many characters are in s
- **s.charAt(i)** (Returns: char) – the character at index i (the first index is 0)
- **s.substring(b, e)** (Returns: String) – characters from index b up to, but not including, e
- **s.substring(b)** (Returns: String) – from index b to the end of s
- **s.indexOf(t)** (Returns: int) – index of the first t in s, or -1 if not found
- **s.contains(t)** (Returns: boolean) – true if t appears somewhere in s
- **s.split(t)** (Returns: String[]) – the pieces of s between the ts: "a,b".split(",") is {"a", "b"}
- **s.trim()** (Returns: String) – a new String with whitespace removed from both ends
- **s.isEmpty()** (Returns: boolean) – true if s has no characters
- **s.toUpperCase()** (Returns: String) – a new String, all upper case
- **s.toLowerCase()** (Returns: String) – a new String, all lower case

- **s.equals(t)** (Returns: boolean) – true if s and t are the same text
- **s.equalsIgnoreCase(t)** (Returns: boolean) – the same, ignoring upper/lower case

Arrays

- **new int[n]** (Type: int[]) – creates an array with n slots, each starting at 0
- **new Thing[n]** (Type: Thing[]) – creates an array with n slots, each starting at null
- **Thing[] things = { new Thing(...), new Thing(...) };** – initializer, only at declaration
- **a[i]** (Type: element type) – the element at index i (the first index is 0)
- **a[i] = x** – stores x into slot i
- **a.length** (Type: int) – the number of elements (a field, so no parentheses)

Type Test

- **p instanceof Thing t** (Returns: boolean) – true if the object p refers to is a Thing (or a subclass of it); when it is, t is that same object as a Thing

Exceptions

- **NumberFormatException** – Thrown by Integer.parseInt(s) or Double.parseDouble(s)
- **IllegalArgumentException** – Thrown when a method is given an argument it refuses
- **FileNotFoundException** – Thrown by new FileReader("data.txt")
- **IOException** – Thrown by reading or writing a file
- **ArrayIndexOutOfBoundsException** – Thrown by array access expressions
- **NullPointerException** – Thrown when a method is called on a variable that is null

File I/O

- **Reading with BufferedReader** (needs import java.io.*; in a class of your own):
  - BufferedReader br = new BufferedReader(new FileReader("data.txt")); – opens data.txt for reading; throws FileNotFoundException if there is no such file
  - br.readLine(); – the next line of the file as a String, or null after the last line
  - br.close(); – closes the reader
- **Writing with PrintWriter**:
  - PrintWriter pw = new PrintWriter(new FileOutputStream("out.txt")); – opens out.txt for writing, replacing any old contents
  - PrintWriter pw = new PrintWriter(new FileOutputStream("out.txt", true)); – opens out.txt for appending: the old contents stay, new lines go at the end
  - pw.println(...); – writes one line to the file
  - pw.print(...); – writes to the file without ending the line
  - pw.printf(f, values); – writes to the file with String.format-style holes: pw.printf("$%.2f%n", 3.0)
  - pw.close(); – closes the writer; until then, nothing is guaranteed to be on disk